

Employee Payroll Application

Complete Postman API Testing Guide — Employee & Department Modules

1. System Configuration & Overview

This guide covers full API testing for both **Employee Module** and **Department Module** in the Spring Boot **EmployeePayrollApp**. The project utilizes Spring Data JPA repositories (EmployeeRepository and DepartmentRepository), Project Lombok annotations, Jakarta validation rules, company email restrictions, and H2 database integration.

Component	Repository Interface	Base Endpoint	Key Validations
Employee Module	EmployeeRepository	http://localhost:8081/api/employees	Company email (blocks @gmail.com), Salary > 0, Valid DOJ
Department Module	DepartmentRepository	http://localhost:8081/api/departments	Required Name, Code, Location; Unique Code
Database / Console	H2 In-Memory DB	http://localhost:8081/h2-console	JDBC: jdbc:h2:mem:payrolldb (User: sa, Pass: empty)

2. All API Endpoints Summary

Module	Method	Endpoint Path	Description	Success Code
Employee	POST	/api/employees	Create employee record	201 Created
Employee	GET	/api/employees	Get all employee records	200 OK
Employee	GET	/api/employees/{id}	Get employee by ID	200 OK
Employee	PUT	/api/employees/{id}	Update employee details	200 OK
Employee	DELETE	/api/employees/{id}	Delete employee record	200 OK
Department	POST	/api/departments	Create department record	201 Created
Department	GET	/api/departments	Get all department records	200 OK
Department	GET	/api/departments/{id}	Get department by ID	200 OK
Department	PUT	/api/departments/{id}	Update department record	200 OK
Department	DELETE	/api/departments/{id}	Delete department record	200 OK

3. SECTION A: Employee Module Postman Test Cases

Test Case 1: Create Employee — Valid Payload (Company Email)

Method: POST | **URL:** http://localhost:8081/api/employees | **Header:** Content-Type: application/json

Request Body:

```
{
  "name": "Alex Morgan",
  "email": "alex.morgan@company.com",
  "dept": "Engineering",
  "salary": 75000.00,
  "doj": "2024-01-15"
}
```

Expected Response:

```
HTTP/1.1 201 Created
{
  "empId": 1,
  "name": "Alex Morgan",
  "email": "alex.morgan@company.com",
  "dept": "Engineering",
  "salary": 75000.0,
  "doj": "2024-01-15"
}
```

Test Case 2: Create Employee — Validation Failure (@gmail.com Prohibited)

Method: POST | **URL:** http://localhost:8081/api/employees

Request Body (Invalid Email):

```
{
  "name": "Jane Smith",
  "email": "jane@gmail.com",
  "dept": "Finance",
  "salary": 65000.00,
  "doj": "2024-02-01"
}
```

Expected Response:

```
HTTP/1.1 400 Bad Request
{
  "timestamp": "2026-08-13T14:35:00.123",
  "status": 400,
  "message": "Validation Failed",
  "details": {
    "email": "Company email is required. @gmail.com is not allowed"
  }
}
```

Test Case 3: Get All Employees & Get Employee by ID

Method: GET | **URL:** http://localhost:8081/api/employees

Expected Response:

```
HTTP/1.1 200 OK
[
  {
    "empId": 1,
    "name": "Alex Morgan",
    "email": "alex.morgan@company.com",
    "dept": "Engineering",
    "salary": 75000.0,
    "doj": "2024-01-15"
  }
]
```

Test Case 4: Update & Delete Employee

PUT URL: http://localhost:8081/api/employees/1 | **DELETE URL:** http://localhost:8081/api/employees/1

Update Request Body:

```
PUT Payload:
{
  "name": "Alex Morgan",
  "email": "alex.m@company.com",
  "dept": "Senior Engineering",
  "salary": 85000.00,
  "doj": "2024-01-15"
}
```

Delete Response:

```
DELETE Response:
HTTP/1.1 200 OK
"Employee deleted successfully with id: 1"
```

4. SECTION B: Department Module Postman Test Cases

Test Case 1: Create Department

Method: POST | **URL:** http://localhost:8081/api/departments | **Header:** Content-Type: application/json

Request Body:

```
{
  "deptName": "Information Technology",
  "deptCode": "IT-101",
  "location": "Building A"
}
```

Expected Response:

```
HTTP/1.1 201 Created
{
  "deptId": 1,
  "deptName": "Information Technology",
  "deptCode": "IT-101",
  "location": "Building A"
}
```

Test Case 2: Get All Departments & Get Department by ID

Method: GET | **URL:** http://localhost:8081/api/departments

Expected Response:

```
HTTP/1.1 200 OK
[
  {
    "deptId": 1,
    "deptName": "Information Technology",
    "deptCode": "IT-101",
    "location": "Building A"
  }
]
```

Test Case 3: Update & Delete Department

PUT URL: http://localhost:8081/api/departments/1 | **DELETE URL:** http://localhost:8081/api/departments/1

Update Request Body:

```
PUT Payload:
{
  "deptName": "Information Technology",
  "deptCode": "IT-101",
  "location": "Building C"
}
```

Delete Response:

```
DELETE Response:
HTTP/1.1 200 OK
"Department deleted successfully with id: 1"
```

5. Execution Instructions for Postman & cURL

1. Start the application by executing `JAVA_HOME=/usr/local/Cellar/openjdk@21/21.0.12/libexec/openjdk.jdk/Contents/Home mvn spring-boot:run` in terminal.
2. Open **Postman**, create a new collection *Employee Payroll App*.
3. Add two folders inside the collection: **Employee API** and **Department API**.
4. Set headers for POST/PUT to Content-Type: application/json.
5. Execute requests and verify status codes (201 Created, 200 OK, 400 Bad Request).